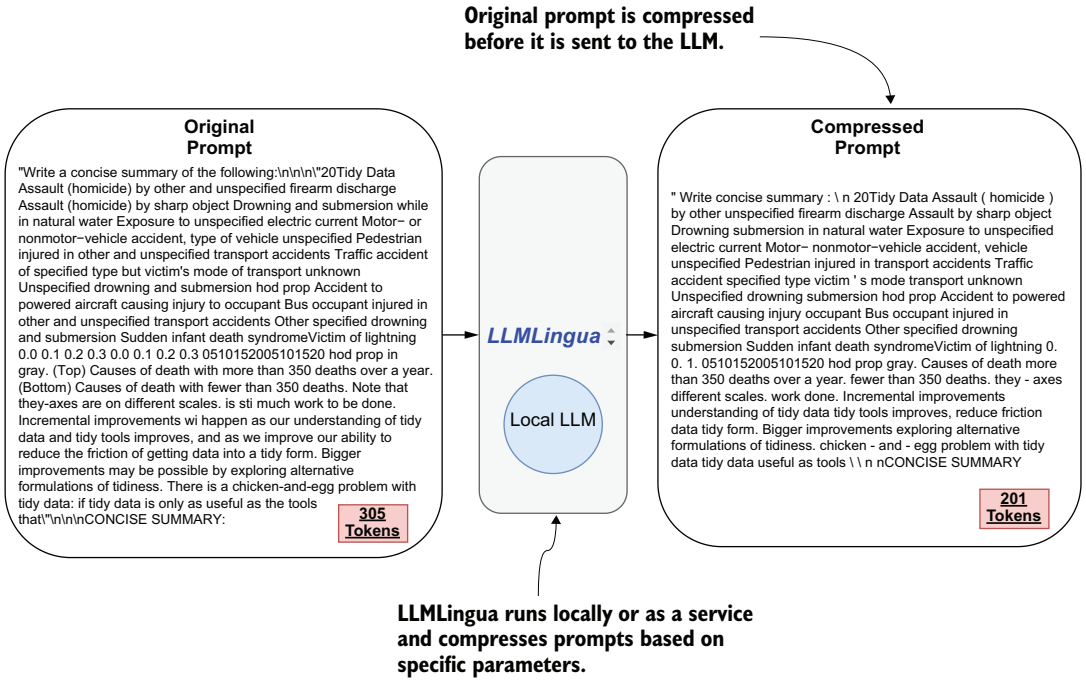


Figure 7.6 The process of compressing a prompt before sending it to an LLM using LLMLingua. The original prompt is compressed into a more concise version, which is then passed to the LLM for processing.

2. *Embedding computation*—We then use a language model to compute embeddings for each chunk. This allows you to represent the semantic content of each chunk as a vector.
3. *Applying k-means clustering*—After creating the chunks, we can apply k-means clustering to group them based on their semantic similarity, which allows us to complete the following optimization step.
4. *Chunk merging*—We then compare the embeddings between chunks and merge any chunks that are deemed semantically similar, based on a distance threshold in the embedding space. This groups the text into semantically coherent sections.

Figure 7.7 shows these steps, and there is also a sample reference JavaScript implementation in the k-means-summarization file inside the ch07/summarization-examples folder.

The document-clustering process is a more complex approach to organizing and preparing text for efficient summarization of large documents. At its core, the `clusterDocuments` function orchestrates three main steps: embedding computation, k-means clustering, and cluster merging. Listing 7.7 focuses on the details of this function.